



MODULE-1
BASIC CONCEPTS

SHORT ANSWER QUESTIONS

1Q. Convert the below octal number to Hexadecimal form $(157)_8$

To Convert Octal to Hexa decimal, we need to convert first to binary, then convert to Hexa decimal

Given: $(157)_8$

i. Conversion of Octal to Binary:

The binary form of 1=001, 5=101, 7=111

$$(157)_8 = (001\ 101\ 111)_2$$

ii. Conversion of Binary to Hexa Decimal:

$$(157)_8 = (001101111)_2$$

$$001101111 = (0000\ 0110\ 1111)$$

The HexaDecimal numbers for 0000=0, 0110=6, 1111=15

$$(0000\ 0110\ 1111)_2 = (6F)_{16}$$

$$(157)_8 = (6F)_{16}$$

2Q. Write about NOR gate with truth table.

NOR GATE

- The output state of the NOR gate will be high (1) when all the inputs are low (0).
- NOR gate returns the complement result of the OR gate. It is basically a combination of two basic logic gates i.e., OR gate and NOT gate.
- The Boolean expression of NOR gate is $X = (A+B)'$

Truth Table		
Input A	Input B	$X = \overline{(A+B)}$
0	0	1
0	1	0
1	0	0
1	1	0

3Q. Minimize the below equation $F=(A+B)(A+C)$ using Boolean theorems.

Given:

$$F=(A+B)(A+C)$$

Expand the expression:

$$F=A(A+C)+B(A+C)$$

$$F=AA+AC+AB+BC$$

Since $AA=A$,

$$F=A+AC+AB+BC$$

$$F=A(1+C)+AB+BC$$

$$F=A+AB+BC$$

Since $1+C=1$

$$F=A(1+B)+BC$$

$$F=A+BC$$

Minimized form

$$F=A+BC$$

4Q. State DeMorgan's Laws of Boolean Algebra.

DeMorgan's laws are important rules used to simplify Boolean expressions.

This law states that the complement of the AND operation of two variables is equal to the OR operation of their complements, and the complement of the OR operation of two variables is equal to the AND operation of their complements.

Mathematically, the laws are expressed as:

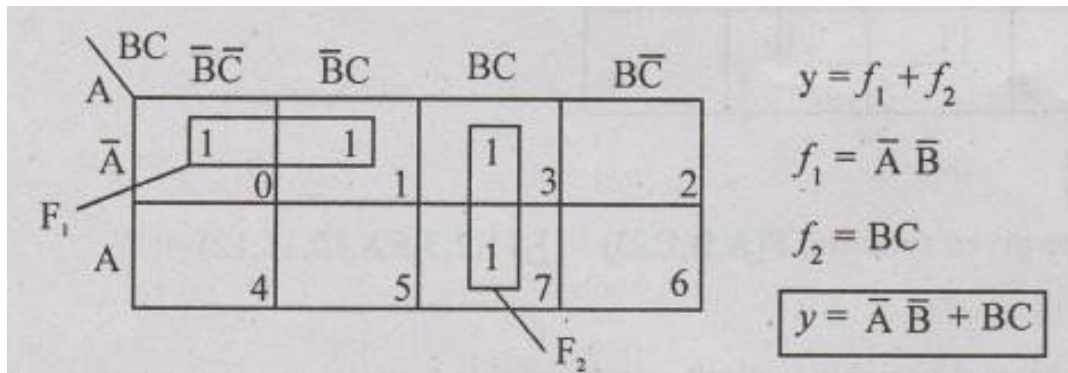
$$\overline{A.B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A}. \overline{B}$$

5Q.Simplify the Boolean function using a 3-variable K-Map

$$F(A,B,C)=\Sigma m(0,1,3,5,7)$$

Here the 3 variable as in the function so we draw 3 variable k-map and fill ones in the given minterms



6Q.Convert $(101101)_2$ and $(110.101)_2$ to Decimal.

Given binary number: $(101101)_2$

Expand using powers of 2: $=1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 =$

$$=1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1$$

$$=32 + 8 + 4 + 1 = 32 + 8 + 4 + 1$$

$$=45$$

$$(101101)_2 = (45)_{10}$$

Given binary number: $(110.101)_2$

Expand using powers of 2: $(110.101)_2 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} =$

$$=4 + 2 + 0.5 + 0 + 0.125$$

$$=6.625$$

$$(110.101)_2 = (6.625)_{10}$$

LONG ANSWER QUESTIONS

1Q. Define Number System. Discuss different types of number systems?

A number system is a method of representing numbers using a set of digits or symbols according to certain rules. It is used to store, process, and communicate data in digital devices. In digital electronics, there are four types of digital number systems are mainly used:

- Binary Number System
- Octal Number System
- Decimal Number System
- Hexadecimal Number System.

Decimal Number System

- The number system that we use in our day-to-day life is the decimal number system.
- Decimal number system has base 10 as it uses 10 digits from 0 to 9.
- In decimal number system, the successive positions to the left of the decimal point represents units, tens, hundreds, thousands and so on.

Example: 156

Binary Number System

- Uses two digits, 0 and 1. Also called base 2 number system
- Each position in a binary number represents a 0 power of the base (2). Example: 2^0
- Last position in a binary number represents an x power of the base (2). Example: 2^x where x represents the last position - 1.

Example: $(101010)_2$

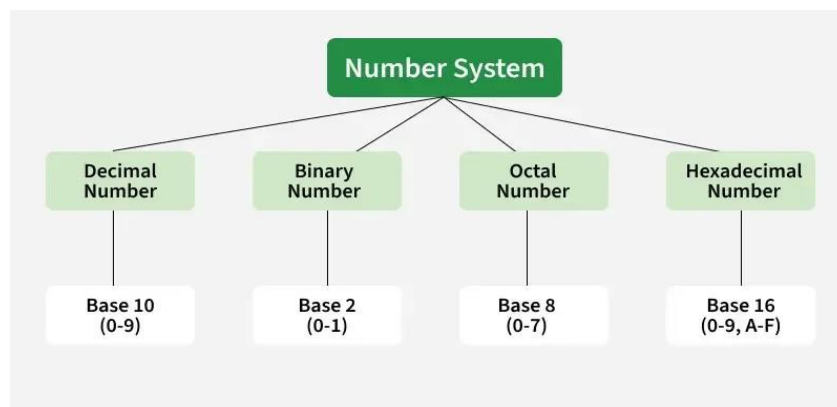
Octal Number System

- Uses eight digits, 0,1,2,3,4,5,6,7. Also called base 8 number system
- Each position in an octal number represents a 0 power of the base (8). Example: 8^0
- Last position in an octal number represents an x power of the base (8). Example: 8^x where x represents the last position - 1.

Example: $(1257)_8$

Hexadecimal Number System

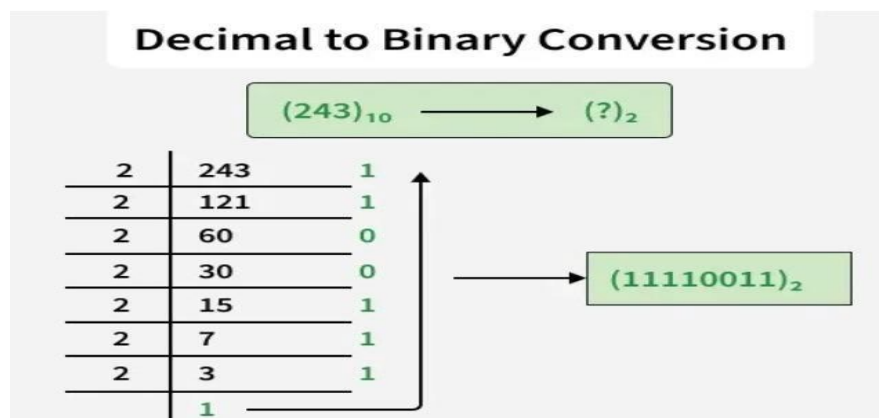
- Uses 10 digits and 6 letters, 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F.
- Letters represents numbers starting from 10. A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.
- Also called base 16 number system.
- Each position in a hexadecimal number represents a 0 power of the base (16). Example 16^0 .
- Last position in a hexadecimal number represents an x power of the base (16). Example 16^x where x represents the last position - 1.



2Q.Convert (243)₁₀ into following number system

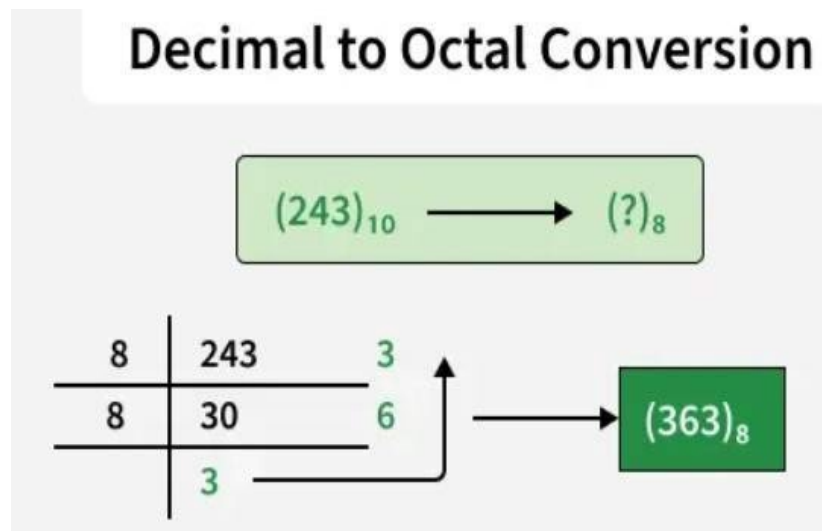
a) Binary number b) Octal number c) Decimal number.

a) **binary number:** Decimal numbers are represented in base 10, but the Binary numbers are of base 2. Hence, to convert the decimal number to a Binary number, the base of that number is to be changed.



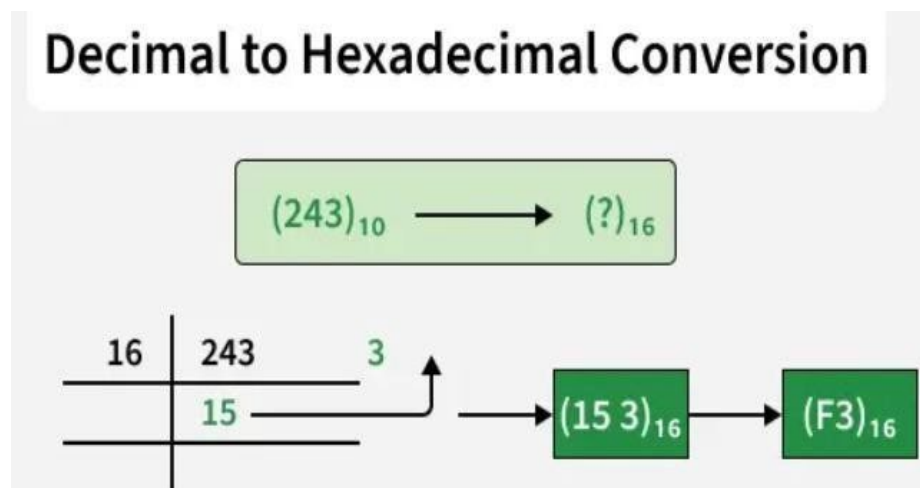
$$(243)_{10} = (11110011)_2$$

b) **Octal number** : Decimal numbers are represented in base 10, but the Octal numbers are of base 8. Hence, to convert the decimal number to a Octal number, the base of that number is to be changed.



$$(243)_{10} = (363)_8$$

c) **Decimal number**: Decimal numbers are represented in base 10, but the Hexadecimal numbers are of base 16. Hence, to convert the decimal number to a Hexadecimal number, the base of that number is to be changed.



$$(243)_{10} = (F3)_{16}$$

3Q.State and explain Boolean Alzebra theorems.

Boolean Algebra is used to analyze and simplify the digital (logic) circuits. It uses only the binary numbers i.e. 0 and 1. It is also called as **Binary Algebra** or **logical Algebra**. Boolean algebra was invented by **George Boole** in 1854.

Rule in Boolean Algebra

Following are the important rules used in Boolean algebra.

- Variable used can have only two values. Binary 1 for HIGH and Binary 0 for LOW.
- Complement of a variable is represented by an overbar (-). Thus, complement of variable B is represented as $\overline{B}$. Thus if $B = 0$ then $\overline{B} = 1$ and $B = 1$ then $\overline{B} = 0$.
- ORing of the variables is represented by a plus (+) sign between them. For example ORing of A, B, C is represented as $A + B + C$.
- Logical ANDing of the two or more variable is represented by writing a dot between them such as A.B.C. Sometime the dot may be omitted like ABC.

Boolean Laws

There are six types of Boolean Laws.

i) Commutative law

Any binary operation which satisfies the following expression is referred to as commutative operation.

$$(i) A.B = B.A \quad (ii) A + B = B + A$$

Commutative law states that changing the sequence of the variables does not have any effect on the output of a logic circuit. This law states that the order in which the logic operations are performed is irrelevant as their effect is the same.

$$(i) (A.B).C = A.(B.C) \quad (ii) (A + B) + C = A + (B + C)$$

ii) Distributive law

Distributive law states the following condition.

$$A.(B + C) = A.B + A.C$$

AND law

These laws use the AND operation. Therefore they are called as **AND** laws.

$$(i) A \cdot 0 = 0$$

$$(ii) A \cdot 1 = A$$

$$(iii) A \cdot A = A$$

$$(iv) A \cdot \bar{A} = 0$$

OR law

These laws use the OR operation. Therefore they are called as **OR** laws.

$$(i) A + 0 = A$$

$$(ii) A + 1 = 1$$

$$(iii) A + A = A$$

$$(iv) A + \bar{A} = 1$$

iii) Inversion law

This law uses the NOT operation. The inversion law states that double inversion of a variable results in the original variable itself.

$$\overline{\overline{A}} = A$$

iv) DeMorgan's law

This law states that the complement of the AND operation of two variables is equal to the OR operation of their complements, and the complement of the OR operation of two variables is equal to the AND operation of their complements. Mathematically, the laws are expressed as:

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

v) Duality Theorem

The Duality Theorem states that every Boolean algebraic expression remains valid if the operators and identity elements are swapped ($\text{AND} \leftrightarrow \text{OR}$, $0 \leftrightarrow 1$). If an expression F is valid, then its dual F^D is also valid, where F^D is obtained by replacing all, 0 with 1, and 1 with 0.

Example: $A + 0 = A$

Dual: $A \cdot 1 = A$

Both are valid Boolean identities.

4Q. a) Discuss about Canonical Forms min term and max term.

b) Convert the Boolean function $F(A,B,C)=A+BC$ Canonical SOP form

Canonical forms provide a unique representation of Boolean functions using minterms or max terms.

Canonical Forms in Boolean Algebra

Canonical forms in Boolean algebra provide a standard way to represent logic functions using minterms or maxterms.

Minterms: A minterm is the product of different Boolean variables, each occurring exactly once, where the output is 1. If the value of the Boolean variable is 1, we take the variable without complementing it and if the value is 0, we take the variable with complement.

Maxterms: A maxterm is the sum of different Boolean variables, each occurring exactly once, where the output is 0. If the value of the Boolean variable is 0, we take the variable without complementing it and if the value is 1, we take the variable with complement.

Consider two Boolean variable A, B and a Boolean function F. The Boolean function F is equal to A XOR B. $F(A, B) = A \oplus B$

Truth Table for F(A, B):

S. No.	A	B	F(A, B)	Minterms (Product Terms)	Maxterms (Sum Terms)
0	0	0	0	No minterm as output is 0	$M_0 = A + B$
1	0	1	1	$m_1 = A'.B$	No maxterm as output is 1
2	1	0	1	$m_2 = A.B'$	No maxterm as output is 1
3	1	1	0	No minterm as output is 0	$M_3 = A' + B'$

Canonical Sum of Products (SOP) Form for the Boolean Function 'F':

$$F(A, B) = m_1 + m_2 = A'.B + A.B'$$

Canonical Product of Sums (POS) Form for the Boolean Function 'F':

$$F(A, B) = M_0 + M_3 = (A + B).(A' + B')$$

Convert to Canonical SOP

Every product term must contain **all variables**.

b) Given:

$$F=A+BC$$

$$F=A(B+B')(C+C')+BC(A+A')$$

$$F=ABC+ABC'+AB'C+AB'C'+ABC+ A'BC$$

$$F = ABC + ABC' + AB'C + AB'C' + ABC + A'BC$$

Remove duplicate ABC:

$$F=A'BC+AB'C'+AB'C+ABC'+ABC$$

Canonical SOP:

$$F=\Sigma m(3,4,5,6,7)$$

5Q.a)Write about K-Map and explain.

b)Simplify the following Boolean expression using the 4-variable K-map.

$$f(A,B,C,D)=\Sigma m(2,3,6,7,8,10,13,15)$$

a)The Karnaugh map (or K-map) is a visual way of detecting redundancy in the SoP. The K-map can be easily used for circuits with 2, 3, or 4 inputs.It consists of an array of cells, each representing a possible combination of inputs.

The cells are arranged to that each cell's input combination differs from adjacent cells by only a single bit.

This is called Gray code ordering – it ensures that physical neighbours in the array are logical neighbours as well. (In other words, neighbouring bit patterns are nearly the same, differing by only 1 bit).

2-input:

a \ b	0	1
0		
1		

3-input:

a \ bc	00	01	11	10
0				
1				

4-input:

ab \ cd	00	01	11	10
00				
01				
11				
10				

Note that the numbers are *not* in binary order, but are arranged so that only a single bit changes between neighbours.

Rules for K-Maps

1. Each cell with a 1 must be included in at least one group.
2. Try to form the largest possible groups.
3. Try to end up with as few groups as possible.
4. Groups may be in sizes that are powers of 2: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, ...
5. Groups may be square or rectangular only (including wrap-around at the grid edges). No diagonals or zig-zags can be used to form a group.
6. The larger a group is, the more redundant inputs there are:
 - i. A group of 1 has no redundant inputs.
 - ii. A group of 2 has 1 redundant input.
 - iii. A group of 4 has 2 redundant inputs.
 - iv. A group of 8 has 3 redundant inputs.
 - v. A group of 16 has 4 redundant inputs.

b)

The SOP K-map representation of the given Boolean function is shown in Figure 2.

CD \ AB		CD			
		00	01	11	10
00	0			1	1
	1			3	2
01	4			1	
	5			7	6
11	12		1	1	
	13			15	14
10	8	1			
	10				1

Figure 2 - SOP K-Map

Explanation:

The simplification of the function is done as per the following steps –

There are no isolated 1s in the K-Map.

The minterm m2 can form a 4-square with m3, m6, and m7. Make it and read it as –

A'C

The minterm m8 can form a 2-square with m10. Make it and read it as –

ABD

The minterm m13 can form a 2-square with m15. Make it and read it as –

AB'D'Type equation here.

Write all the product in SOP form.

So the simplified SOP expression is,

$$f(A,B,C,D)=A'C+ABD+AB'D'$$

6Q. Implement basic logic gates using NAND and NOR Gates.

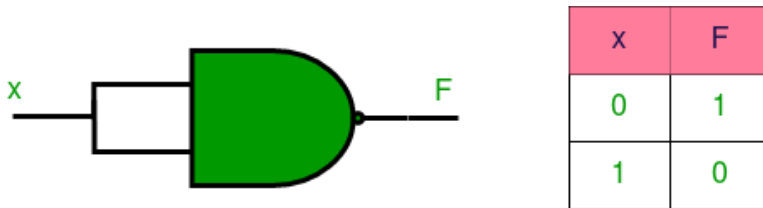
In Boolean Algebra, the **NAND** and **NOR** gates are called **universal gates** because any digital circuit can be implemented by using any one of these two i.e. any logic gate can be created using NAND or NOR gates only.

Implementation of NOT Gate using Universal gates.

Implementation using NAND Gates

A single NAND gate with 'a' and 'b' inputs connected respond to a NOT gate. The NOT gate function is achieved as the result of NAND operation because the input signal is inverted.

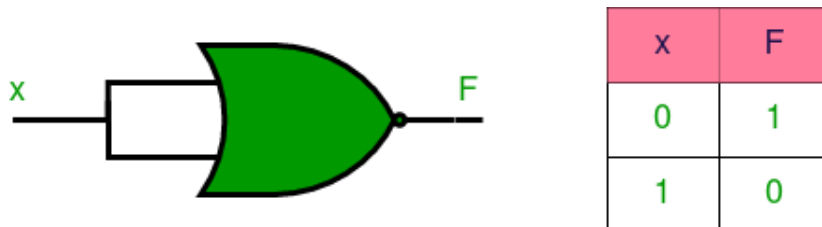
Implementation of NOT gate using a single NAND gate as shown in the picture below:



Implementation Using NOR Gates

A single NOR gate with joining of both inputs perform the function of NOT gate. The input is negated and the output thus obtained is the inversion that is characteristic of an NOT gate.

Implementation of NOT gate using a single NOR gate as shown in the picture below:

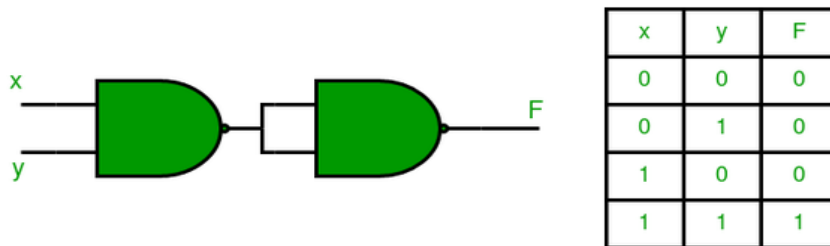


Implementation of AND Gate using Universal Gates

Implementation using NAND Gates

The AND gate is implemented using two NAND gates where the output is HIGH only when both inputs are HIGH. The first NAND gate performs a standard NAND functions, the second NAND gate has both inputs tied to the output of the initial NAND gate as shown below. This particular configuration flips the output thus emulating the behavior of an AND gate.

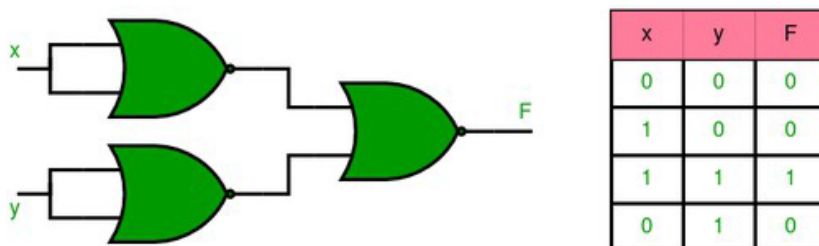
The AND gate can be implemented by using two NAND gates in the below fashion:



Implementation using NOR Gates

AND gate is implemented using three NOR gates and the setting of this Gate is referred to as 111. The inputs are then transformed with the use of a NOR gate and then they are negated one more time by using two more NOR gates. Together, these create the AND function, meaning that all of the conditions set have to be met in order to pass.

Implementation of AND gate using only NOR gates as shown below

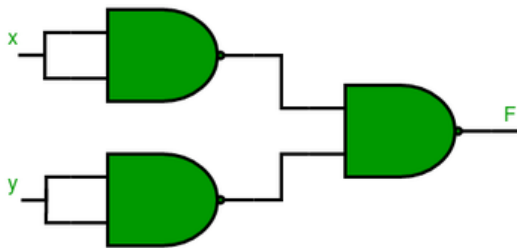


Implementation of OR Gate using Universal

Gates Implementation Using NAND Gates

It is implemented using three NAND gates. The inputs are first inverted using two NAND gates having their inputs probed in parallel and then the inverted outputs are connected with the input of third NAND gate. The last NAND gate effectively performs the inverted signals to give the required OR gate function.

The OR gate can be implemented using the NAND gate as below

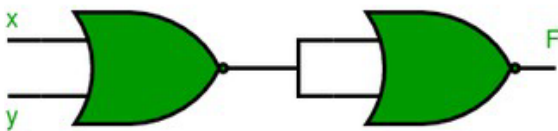


x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

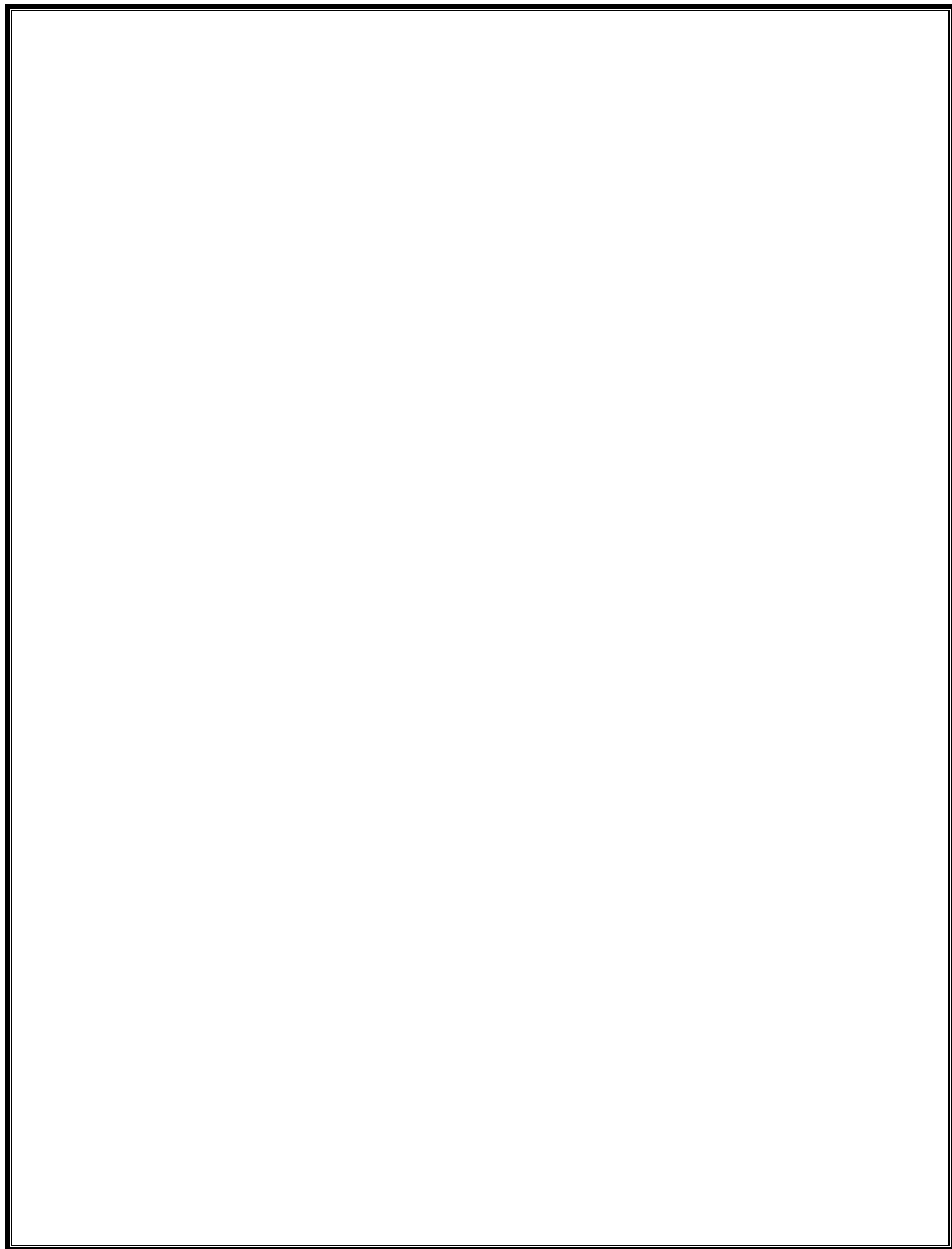
Implementation using NOR Gates

The OR gate is created by using an inverse of the output of the NOR operation. An inverted form of the inputs is provided directly into the NOR gate while the output is provided in NOR gate using fixed inputs. This inversion gives the OR gate logic as shown below.

Implementation of OR gate using two NOR gates as shown in the picture below:



x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



MODULE -2

COMBINATIONAL LOGIC CIRCUITS **SHORT ANSWER QUESTIONS**

1Q) Define BCD Code and mention its Advantages and Disadvantages?

Binary Coded Decimal (BCD) code

In this code each decimal digit is represented by a 4-bit binary number. BCD is a way to express each of the decimal digits with a binary code. In the BCD, with four bits we can represent sixteen numbers (0000 to 1111). But in BCD code only first ten of these are used (0000 to 1001). The remaining six code combinations i.e. 1010 to 1111 are invalid in BCD.

Decimal	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Advantages of BCD Codes

- It is very similar to decimal system.
- We need to remember binary equivalent of decimal numbers 0 to 9 only.

Disadvantages of BCD Codes

- The addition and subtraction of BCD have different rules.

- The BCD arithmetic is little more complicated.

2Q) Define Half Adder and draw its Truth Table?

Half adder is a combinational logic circuit with two inputs and two outputs. The half adder circuit is designed to add two single bit binary number A and B. It is the basic building block for addition of two **single** bit numbers. This circuit has two outputs **carry** and **sum**.

Block diagram



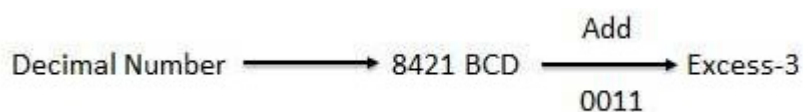
Inputs		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Truth table

3Q) what is meant by Excess-3 code and give one example?

Excess-3 code

The Excess-3 code is also called as XS-3 code. It is non-weighted code used to express decimal numbers. The Excess-3 code words are derived from the 8421 BCD code words adding (0011)₂ or (3)₁₀ to each code word in 8421. The excess-3 codes are obtained as follows –



Example

Decimal	BCD				Excess-3			
	8	4	2	1	BCD + 0011			
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

4Q) Define code converter and its types?

Definition

A **code converter** is a **combinational logic circuit** that converts data from one binary code to another. The output depends only on the current input and does not require memory or clock signals.

Common Types of Code Converters

1. Binary to Gray Code Converter
2. Gray to Binary Code Converter
3. BCD to Excess-3 Code Converter
4. Excess-3 to BCD Code Converter

5Q) what is the purpose of Digital Comparator?

The purpose of a **Digital Comparator** is to compare a set of variables or unknown numbers, for example A (A1, A2, A3, An, etc) against that of a constant or unknown value such as B (B1, B2, B3, Bn, etc) and produce an output condition or flag depending upon the result of the comparison. For example, a magnitude comparator of two 1-bits, (A and B) inputs would produce the following three output conditions when compared to each other.

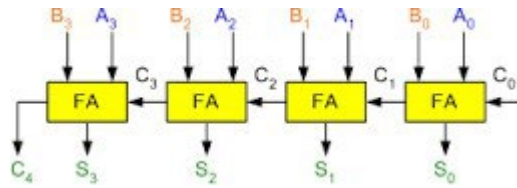
$$A > B, \quad A = B, \quad A < B$$

Which means: A is greater than B, A is equal to B, and A is less than B

This is useful if we want to compare two variables and want to produce an output when any of the above three conditions are achieved.

6Q) what is the drawback of Ripple Carry Adder. How to overcome it?

In ripple carry adders, the carry propagation time is the major speed limiting factor.



Most other arithmetic operations, e.g. multiplication and division are implemented using several add/subtract steps. Thus, improving the speed of addition will improve the speed of all other arithmetic operations. One widely used approach employs the principle of carry look-ahead solves this problem by calculating the carry signals in advance, based on the input signals.

This type of [adder circuit](#) is called as carry look-ahead adder (CLA adder). It is based on the fact that a carry signal will be generated in two cases:

- (1) when both bits A_i and B_i are 1, or
- (2) when one of the two bits is 1 and the carry-in (carry of the previous stage) is 1.

LONG ANSWER QUESTIONS

1Q.Explain Binary to Gray code converter and derive Boolean expression using truth table?

Converting Binary to Gray Code

Let b_0, b_1, b_2 and b_3 be the bits representing the binary numbers, where b_0 is the LSB and b_3 is the MSB, and Let g_0, g_1, g_2 , and g_3 be the bits representing the gray code of the binary numbers, where g_0 is the LSB and g_3 is the MSB.

The truth table for the conversion is

Binary (B3 B2 B1 B0)	Gray Code (G3 G2 G1 G0)
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101
1010	1111
1011	1110
1100	1010
1101	1011

Binary (B3 B2 B1 B0)	Gray Code (G3 G2 G1 G0)
1110	1001
1111	1000

To find the corresponding digital circuit, we will use the K-Map technique for each of the gray code bits as output with all of the binary bits as input.

K-map for g_0 :

		b1,b0			
		00	01	11	10
b3,b2	00	0	1	0	1
	01	0	1	0	1
	11	0	1	0	1
	10	0	1	0	1

K-map for g_1 :

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	1	1
	01	1	1	0	0
	11	1	1	0	0
	10	0	0	1	1

K-map for g_2 :

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

K-map for g_3 :

		b1,b0			
		00	01	11	10
b3,b2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

Corresponding Minimized Boolean Expressions and Digital Circuit for Gray Code Bits

$$g_0 = b_0 b_1' + b_1 b_0' = b_0 \oplus b_1$$

$$g_1 = b_2 b_1' + b_1 b_2' = b_1 \oplus b_2$$

$$g_2 = b_2 b_3' + b_3 b_2' = b_2 \oplus b_3$$

Boolean expression for conversion of binary to gray code for n-bit :

$$g_n = b_n$$

$$g_{n-1} = b_n \text{ XOR } b_{n-1}$$

$$g_1 = b_2 \text{ XOR } b_1$$

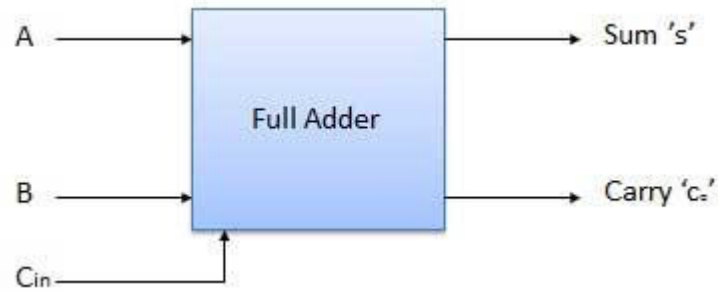
2Qa).Explain Full Adder Circuit with truth table and logic diagram?

b)How to convert BCD to Excess-3 and give one example?

Full Adder

Full adder is developed to overcome the drawback of Half Adder circuit. It can add two one-bit numbers A and B, and carry c. The full adder is a three input and two output combinational circuit.

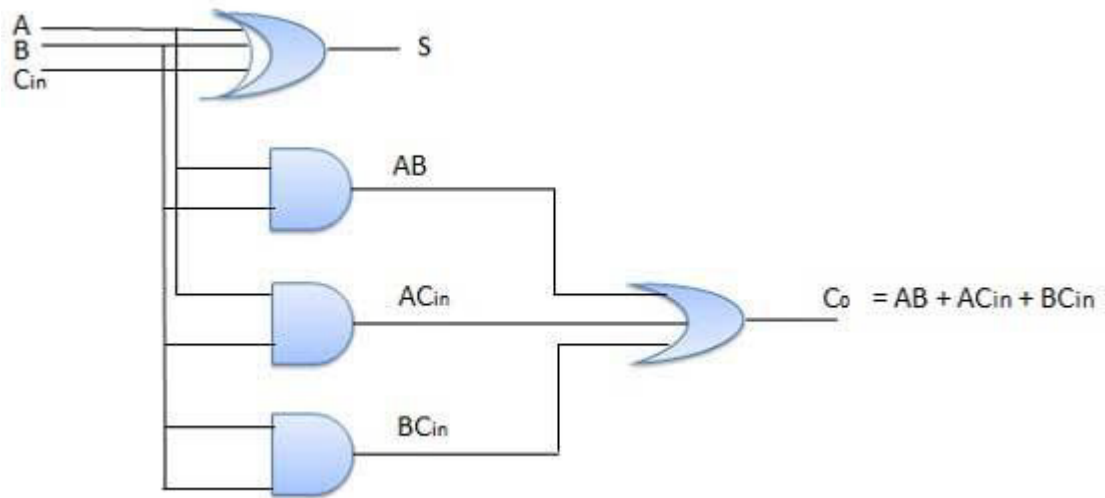
Block diagram



Truth Table

Inputs			Output	
A	B	C _{in}	S	Co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit Diagram



b) BCD to Excess-3

- **Step 1** -- Convert BCD to decimal.
- **Step 2** -- Add $(3)_{10}$ to this decimal number.
- **Step 3** -- Convert into binary to get excess-3 code.

Example – convert $(1001)_{BCD}$ to Excess-3.

Step 1 – Convert to decimal

$$(1001)_{BCD} = 9_{10}$$

Step 2 – Add 3 to decimal

$$9_{10} + (3)_{10} = (12)_{10}$$

Step 3 – Convert to Excess-3

$$(12)_{10} = (1100)_2$$

Result

$$(1001)_{BCD} = (1100)_{XS-3}$$

3Q) Explain the operation of BCD Adder with block diagram?

A BCD adder is a circuit that adds two BCD digits in parallel and produces a sum BCD digit and a carry out bit.

The maximum sum result of a BCD input adder can be 19. As maximum number in BCD is 9 and may be there will be a carry from previous stage also, so $9 + 9 + 1 = 19$

The following truth table shows all the possible sum results when two BCD digits are added.

Z_3	Z_2	Z_1	Z_0	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Correction is done through the addition of 6 to the result to skip the six invalid values as shown in the truth table by yellow color.

Consider the given examples of non-BCD sum result and its correction.

Non-BCD	→	0101 = 5
BCD correction	→	$\begin{array}{r} 0101 = 5 \\ +0111 = +7 \\ \hline 1100 = 12 \end{array}$
In BCD	→	$\begin{array}{r} 1100 = 12 \\ +0110 = +6 \\ \hline 1\ 0010 = 1\ 2 \end{array}$

Non-BCD	→	1001 = 9
BCD correction	→	$\begin{array}{r} 1001 = 9 \\ +0110 = +6 \\ \hline 1111 = 15 \end{array}$
In BCD	→	$\begin{array}{r} 1111 = 15 \\ +0110 = +6 \\ \hline 1\ 0101 = 1\ 5 \end{array}$

Non-BCD	→	1001 = 9
BCD correction	→	$\begin{array}{r} 1001 = 9 \\ +1001 = +9 \\ \hline 1\ 0010 = 18 \end{array}$
In BCD	→	$\begin{array}{r} 1\ 0010 = 18 \\ +0110 = +6 \\ \hline 1\ 1000 = 1\ 8 \end{array}$

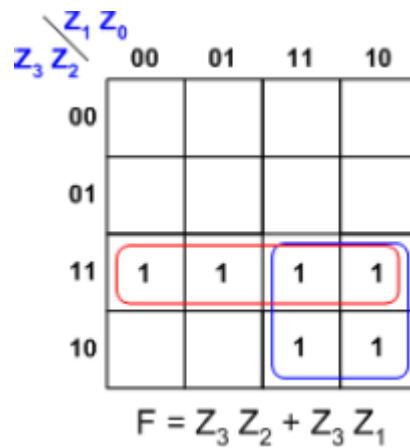
A BCD adder is a circuit that adds two BCD digits in parallel and produces a sum BCD digit and a carry out bit.

The maximum sum result of a BCD input adder can be 19. As maximum number in BCD is 9 and may be there will be a carry from previous stage also, so $9 + 9 + 1 = 19$

The following truth table shows all the possible sum results when two BCD digits are added.

Dec	CO	Z ₃	Z ₂	Z ₁	Z ₀	F
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	1	0	0
3	0	0	0	1	1	0
4	0	0	1	0	0	0
5	0	0	1	0	1	0
6	0	0	1	1	0	0
7	0	0	1	1	1	0
8	0	1	0	0	0	0
9	0	1	0	0	1	0
10	0	1	0	1	0	1
11	0	1	0	1	1	1
12	0	1	1	0	0	1
13	0	1	1	0	1	1
14	0	1	1	1	0	1
15	0	1	1	1	1	1
16	1	0	0	0	0	1
17	1	0	0	0	1	1
18	1	0	0	1	0	1
19	1	0	0	1	1	1

The logic circuit that checks the necessary BCD correction can be derived by detecting the condition where the resulting binary sum is 01010 through 10011 (decimal 10 through 19). It can be done by considering the shown truth table, in which the function F is true when the digit is not a valid BCD digit. It can be simplified using a 5-variable K-map. But detecting values 1010 through 1111 (decimal 10 through 15) can also be done by using a 4-variable K-map as shown in the figure.

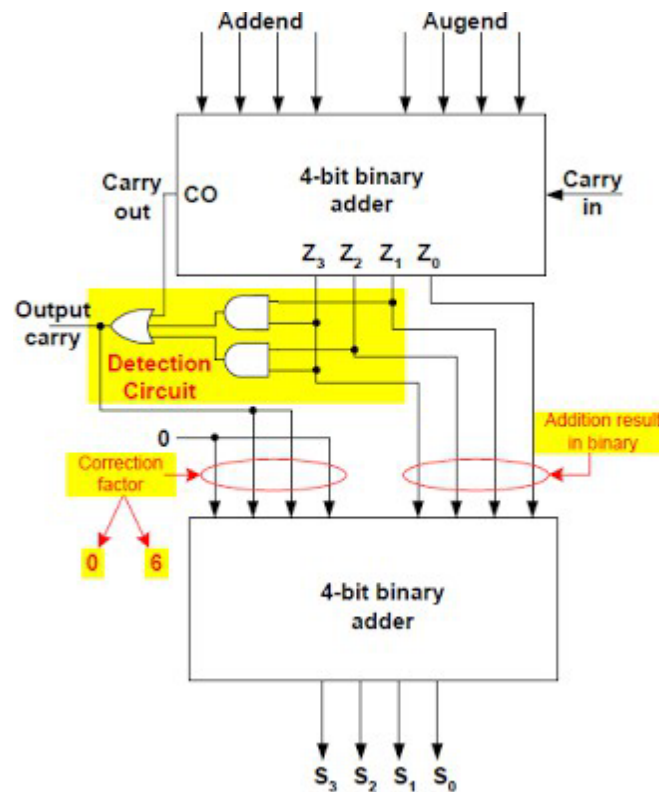


Values greater than 1111, i.e., from 10000 through 10011 (decimal 16 through 19) can be detected by the carry out (CO) which equals 1 only for these output values. So, $F = CO = 1$ for these values. Hence, F is true when CO is true OR when $(Z_3 Z_2 + Z_3 Z_1)$ is true.

Thus, the correction step (adding 0110) is performed if the following function equals 1:

$$F = CO + Z_3 Z_2 + Z_3 Z_1$$

The circuit of the BCD adder will be as shown in the figure.



The two BCD digits, together with the input carry, are first added in the top 4-bit binary adder to produce the binary sum. The bottom 4-bit binary adder is used to add the correction factor to the binary result of the top binary adder.

Note:

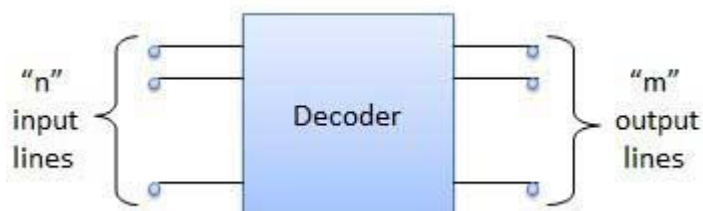
1. When the Output carry is equal to zero, the correction factor equals zero.
2. When the Output carry is equal to one, the correction factor is 0110.

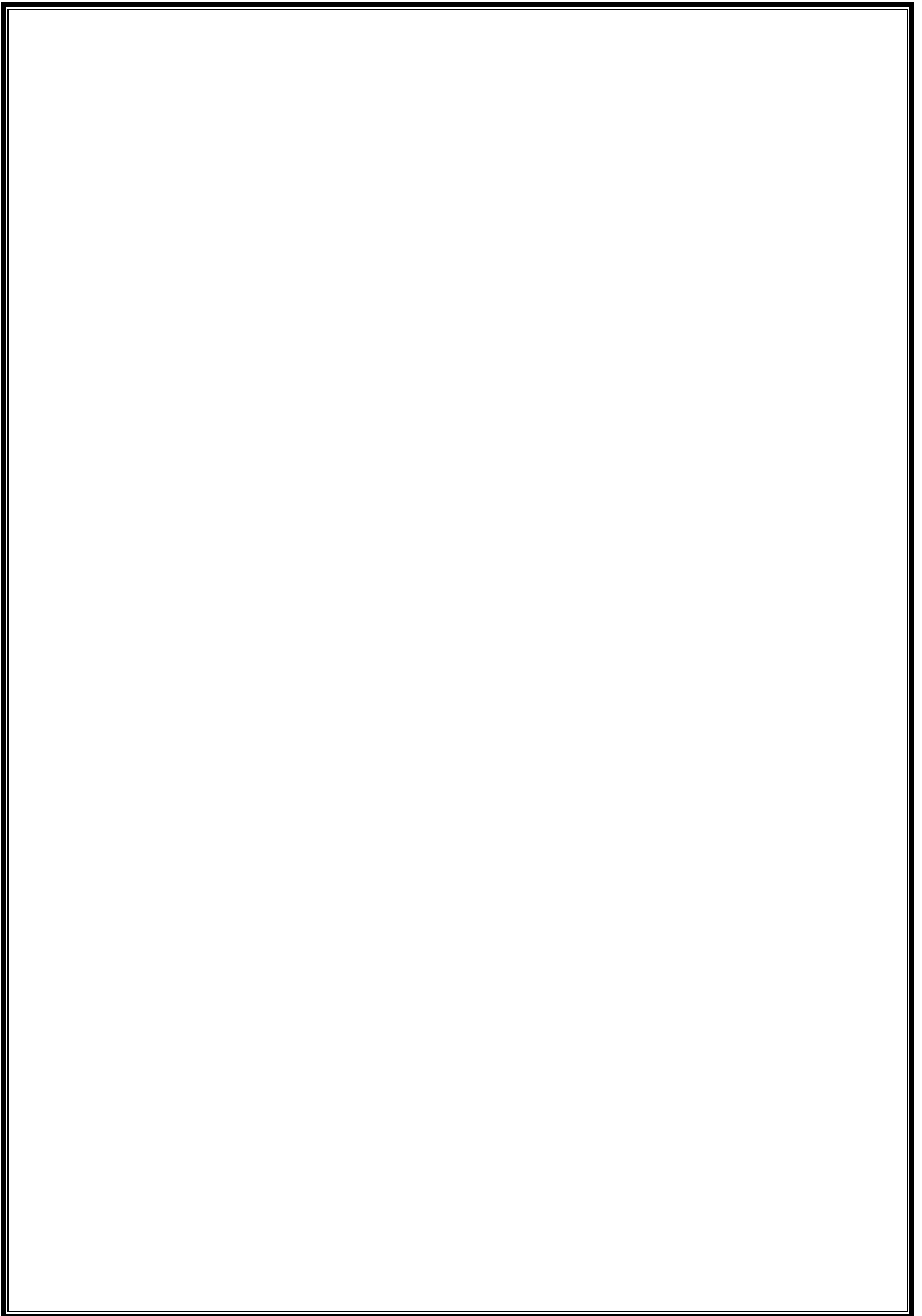
The output carry generated from the bottom binary adder is ignored, since it supplies information already available at the output-carry terminal.

A decimal parallel adder that adds n decimal digits needs n BCD adder stages. The output carry from one stage must be connected to the input carry of the next higher-order.

4Q) Explain about the operation of Decoder ?

A decoder is a combinational circuit. It has n input and to a maximum $m = 2^n$ outputs. Decoder is identical to demultiplexer without any data input. It performs operations which are exactly opposite to those of an encoder.

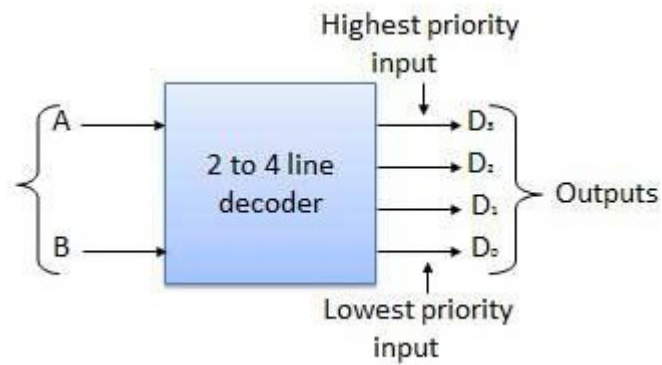




2 to 4 Line Decoder

The block diagram of 2 to 4 line decoder is shown in the fig. A and B are the two inputs where D through D are the four outputs. Truth table explains the operations of a decoder. It shows that each output is 1 for only a specific combination of inputs.

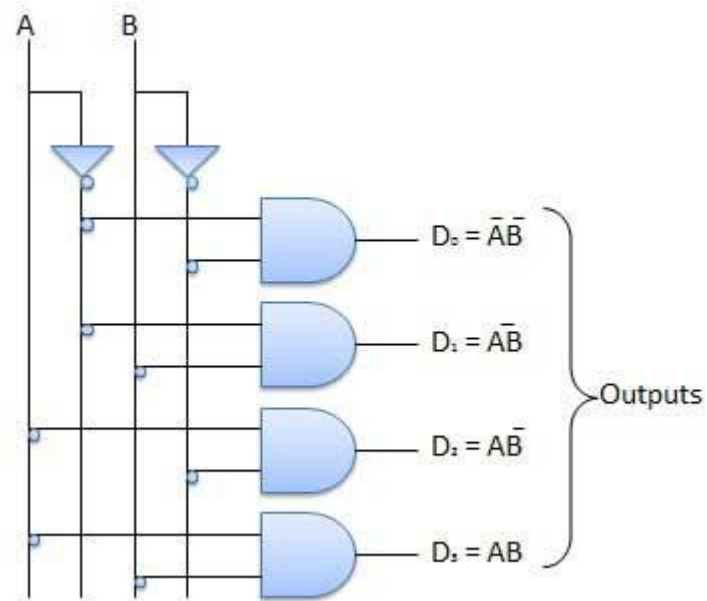
Block diagram



Truth Table

Inputs		Output			
A	B	D ₀	D ₁	D ₂	D ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Logic Circuit



in this topic we will try to discuss about Combinational Logic Implementation full adder circuit with a decoder and two OR Gates.

To do this at first we need to review the truth table of Full Adder circuit.

Inputs			Outputs	
a	b	c	C	S
0	0	0	0	0
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0
0	1	1	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

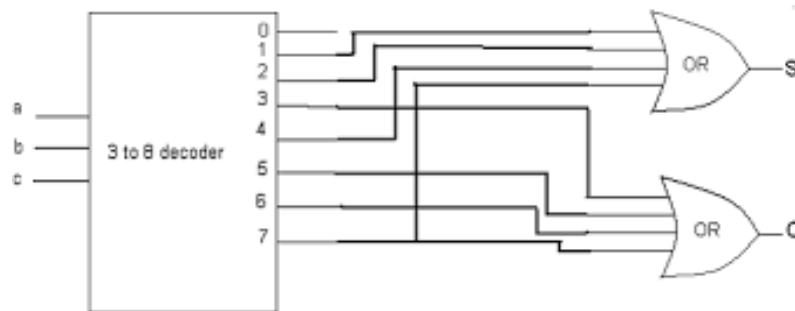
Truth table of Full Adder Circuit

From the truth table we have been found that $S(a,b,c) = \sum(1,2,4,7)$, $C(a,b,c) = \sum(3,5,6,7)$

As there are three inputs and eight min-terms, so have to use 3 to 8 line decoder. The generates the eight min-terms for a, b, c.

The OR Gate for output S forms the sum of min-terms 1, 2, 4 and 7. The OR Gate output C forms the sum of min-terms 3, 5, 6 and 7.

Circuit Diagram



A function with a long list of min-terms requires an OR Gate with a large number of inputs. A function F having a list of K min-terms can be expressed in its complemented form F' with $2^n - K$ min-terms. If the number of min-terms in a function is greater than $2^n/2$, then F' can be expressed with fewer min-terms than required for F . In such a case, it is suitable to use a NOR Gate to sum the min-terms of F' . The output of the NOR Gate will generate the normal output F .

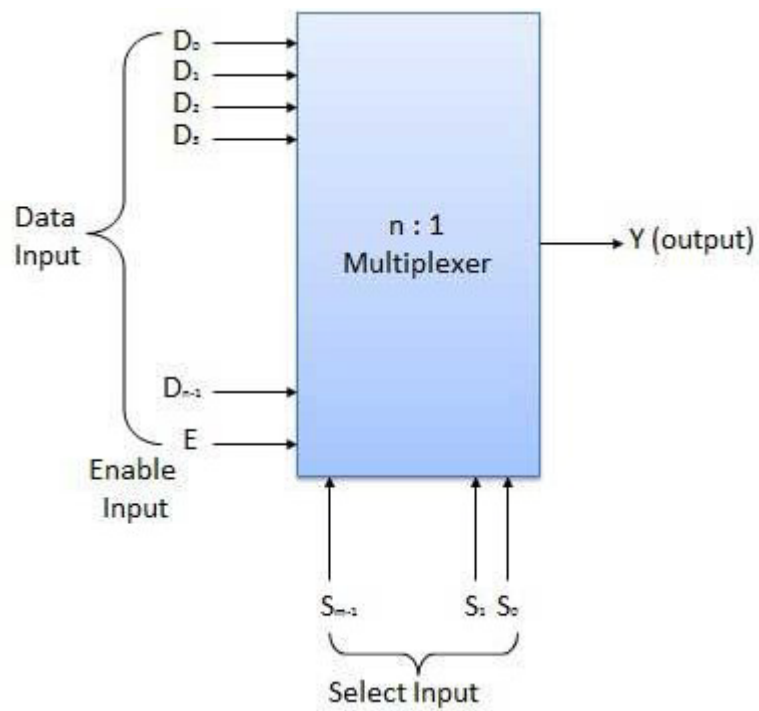
The decoder method can be used to implement any combinational circuit. It is necessary to implementing with comparing to all other possible implementations to determine the solution.

5Q) Explain about Multiplexer and Demultiplexer?

Multiplexers

Multiplexer is a special type of combinational circuit. There are n -data inputs, one output and m select inputs with $2^m = n$. It is a digital circuit which selects one of the n data inputs and routes it to the output. The selection of one of the n inputs is done by the selected inputs. Depending on the digital code applied at the selected inputs, one out of n data sources is selected and transmitted to the single output Y . E is called the strobe or enable input which is useful for the cascading. It is generally an active low terminal that means it will perform the required operation when it is low.

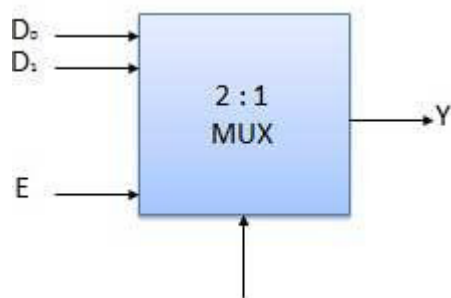
Block diagram



Multiplexers come in multiple variations

- 2 : 1 multiplexer
- 4 : 1 multiplexer
- 16 : 1 multiplexer
- 32 : 1 multiplexer

Block Diagram



Truth Table

Enable	Select	Output
E	S	Y
0	x	0
1	0	D ₀
1	1	D ₁

x = Don't care

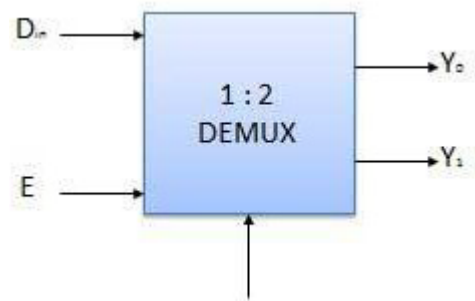
Demultiplexers

A demultiplexer performs the reverse operation of a multiplexer i.e. it receives one input and distributes it over several outputs. It has only one input, n outputs, m select input. At a time only one output line is selected by the select lines and the input is transmitted to the selected output line. A de-multiplexer is equivalent to a single pole multiple way switch as shown in fig.

Demultiplexers come in multiple variations.

- 1 : 2 demultiplexer
- 1 : 4 demultiplexer
- 1 : 16 demultiplexer
- 1 : 32 demultiplexer

Block diagram



Truth Table

Enable	Select	Output	
E	S	Y0	Y1
0	x	0	0
1	0	0	D_{in}
1	1	D_{in}	0

x = Don't care

6) Define Combinational logic circuits and explain its classification?

Combinational circuit is a circuit in which we combine the different gates in the circuit, for example encoder, decoder, multiplexer and demultiplexer. Some of the characteristics of combinational circuits are following –

- The output of combinational circuit at any instant of time, depends only on the levels present at input terminals.
- The combinational circuit do not use any memory. The previous state of input does not have any effect on the present state of the circuit.

- A combinational circuit can have an n number of inputs and m number of outputs.

Block diagram



Classification of Combinational Circuits

Combinational circuits are generally grouped into three main types based on their functions:

1. **Arithmetic and Logic Circuits:** These circuits handle mathematical and logical operations, such as addition, subtraction, and comparison of binary numbers.
2. **Data Routing Circuits:** These circuits manage the flow of data within a system by directing and controlling data transfer between different components.
3. **Code Conversion Circuits:** These circuits are responsible for converting information from one code or number system to another, like converting binary values into decimal or other formats.

MODULE -3

SYNCHRONOUS SEQUENTIAL CIRCUITS

1Q) Define FLIP-FLOP.

It is the basic storage element in sequential logic. Flip-flops in digital electronics are circuits with two stable states that can be used to store binary data. The stored data can be changed by applying varying inputs.

Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems. Both are used as data storage elements.

2Q) Mention applications of Asynchronous Sequential Circuit ?

Asynchronous sequential circuits are widely used in digital electronics due to their speed and efficiency. Here are some of the key applications:

1.Counters: Asynchronous circuits are used in digital counters to perform operations like frequency division and event counting.

2.Registers: Registers are memory elements that store and retrieve data, and asynchronous circuits are often used in register designs.

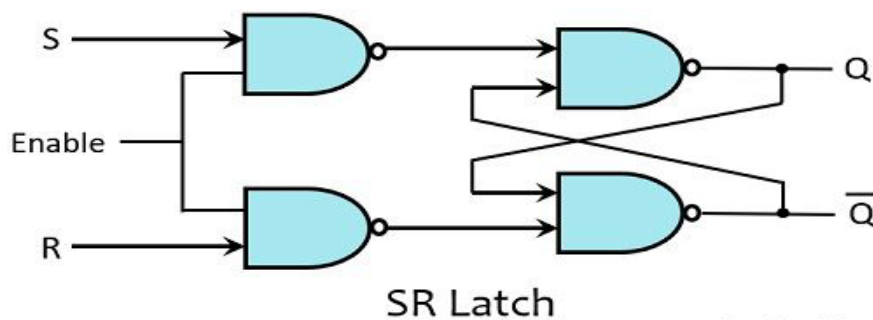
3.Memory Devices: Flipflops and other memory elements are crucial components in digital memory devices, and asynchronous circuits are employed in their design.

4.Control Circuits: Asynchronous circuits are used in control circuits to manage the operation of machines based on predefined logics.

3Q) DEFINE LATCH?

Latches are the sequential logic circuits that are designed using logic gates and are used to store binary information. These are level triggered thus operate when they are enabled. These are also known as **bistable multivibrator** because of having two stable states.

It operates in a way that when the enable pin is active, then every time with the change in input, the output gets changed. This means with the change in input, the stored information also changes. The figure below represents the SR Latch,



Circuit Globe

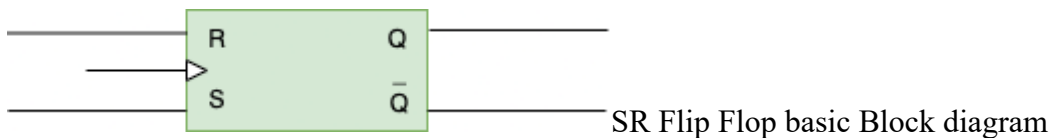
LONG ANSWER QUESTIONS

1Q. Explain the operation of SR FLIP-FLOP?

SR flip flop, also known as SR latch is the basic and simplest type of flip flop. It is a single bit storage element. It has only two logic gates. The output of each gate is connected to the input of another gate.

The state of the SR flip flop is determined by the condition of the output Q. If its value is 1, then the state is said to be SET and if $Q = 0$, the state is said to be RESET. Hence it is called SR flip flop. Basic Block Diagram of SR Flip Flop

The basic block diagram contains S and R inputs, and between them is clock pulse, Q and Q' is the complemented outputs.



Working of SR Flip Flop

- Case 1 : Let's say, $S=0$ and $R=0$, then output of both AND gates will be 0 and the value of Q and Q' will be same as their previous value, i.e, Hold state.
- Case 2 : Let's say, $S=0$ and $R=1$, then output of both AND gates will be 1 and 0, correspondingly the value of Q will be 0 as one of input is 1 and it is a NOR gate so it will ultimately gives 0, hence Q gets 0 value, similarly Q' will be 1.
- Case 3 : Let's say, $S=1$ and $R=0$, then output of both AND gates will be 0 and 1, correspondingly the value of Q' will be 0 as one of input to NOR gate is 1, so output will be 0 ultimately and this 0 value will go as input to upper NOR gate, and hence Q will become 1.
- Case 4 : Let's say, $S=1$ and $R=1$, then output of both AND gates will be 1 and 1 which is invalid, as the outputs should be complement of each other.

Truth Table of SR Flip Flop

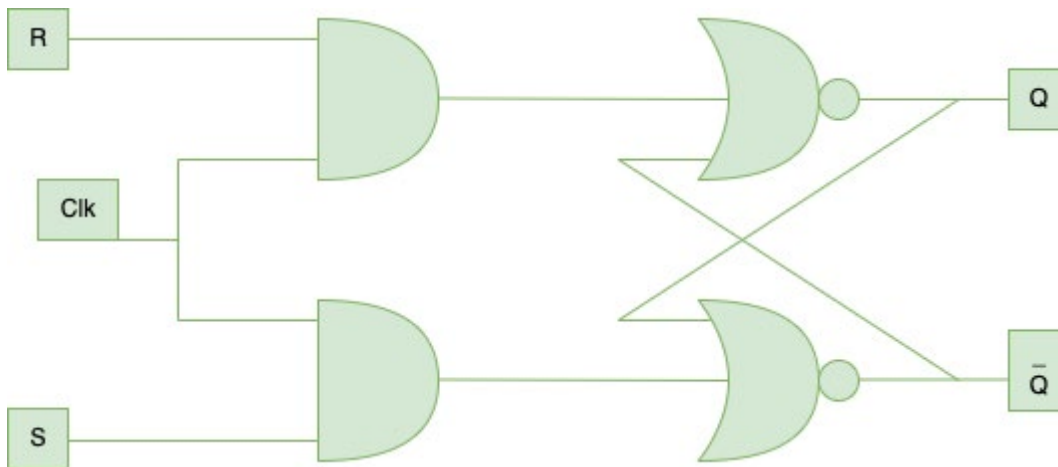
S	R	Q _{n+1}	State
0	0	Q _n	Hold
0	1	0	Reset
1	0	1	Set
1	1	X	Invalid

Here, **S** is the Set input, **R** is the reset input, **Q_{n+1}** is the next state and **State** tells in which state it enters

Function Table of SR Flip Flop

S	R	Q _n	Q _{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Invalid
1	1	1	Invalid

Here, **S** is the Set input, **R** is the reset input, **Q_n** is the current state input and **Q_{n+1}** is the next state outputs.



SR Flip Fop using two NOR and two AND Gates

2Q. Explain about D-LATCH?

A D latch stores a single bit of data and transfers the input D to the output Q only when the Enable (E) signal is HIGH; when E is LOW, the output retains its previous value.

Basic Operation

A D latch is a digital memory device with two main inputs: Data (D) and Enable (E), and two outputs: Q and $\bar{Q}$ (complement of Q). The D input carries the data to be stored, while the Enable input controls when the data is allowed to pass to the output. The latch is level-sensitive, meaning it responds to the level of the Enable signal rather than a clock edge.

Enable HIGH ($E = 1$): The output Q follows the D input, making the latch "transparent". Any change in D immediately reflects at Q.

Enable LOW ($E = 0$): The output Q holds its previous value, effectively storing the last data present on D.

Truth Table

EN (Enable)	D (Data)	Q (Output)	Description
0	0	Q_prev	Latch is disabled; output retains previous state
0	1	Q_prev	Latch is disabled; output retains previous state
1	0	0	Latch is enabled; output follows D input (reset)
1	1	1	Latch is enabled; output follows D input (set)

Explanation:

When $EN = 0$, the D latch is **not enabled**, so the output **Q does not change** regardless of the D input. This allows the latch to "remember" its previous state (Q_{prev}) and act as a memory element.

When $EN = 1$, the latch is **transparent**, meaning the output **Q directly follows the D input**. If $D = 0$, Q is reset to 0; if $D = 1$, Q is set to 1

Gated D Latch

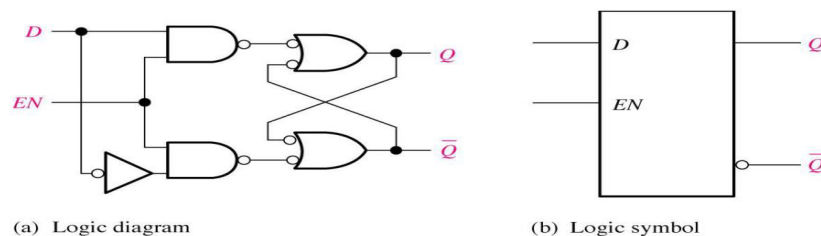


Figure 8-10 A gated D latch.

14

3Q. Write the differences between FLIP-FLOP and LATCH?

Difference Between Flip-Flop and Latch

Given Below are the Difference Between Flip-Flop and Latch

Flip-Flop	Latch
Flip-flop is a bistable device i.e., it has two stable states that are represented as 0 and 1.	Latch is also a bistable device whose state are also represented as 0 and 1.
It checks the inputs but changes the output only at times defined by the clock signal or any other control signal.	It checks the inputs continuously and responds to the changes in input immediately.
It is a edge triggered device.	It is a level triggered device.

Flip-Flop	Latch
Gates like <u>NOR</u> , <u>NOT</u> , <u>AND</u> , <u>NAND</u> are building blocks of flip flops.	These are also made up of gates.
They are classified into asynchronous or synchronous flipflops.	There is no such classification in latches
It forms the building blocks of many sequential circuits like <u>counters</u> .	These can be used for the designing of sequential circuits but are not generally preferred.
Flip-flop always have a clock signal	Latches doesn't have a clock signal
Flip-flop can be build from Latches	Latches can be build from gates
Ex: <u>D Flip-flop</u> , <u>JK Flip-flop</u>	Ex: SR Latch, D Latch